

Middleware in Expressjs

1. Ví dụ thực tế: Logger + Auth + Validation

Chúng ta sẽ tạo thêm một middleware đơn giản để ghi lại nhật ký (Log) mỗi khi có ai đó gọi API.

Bước 1: Tạo Logger Middleware

File: src/middleware/logMiddleware.js

```
const logRequest = (req, res, next) => {  
  const method = req.method; // GET, POST, DELETE...  
  const url = req.originalUrl; // /api/courses  
  const time = new Date().toISOString();  
  
  console.log(`[${time}] ${method} ${url}`);  
  
  // QUAN TRỌNG: Phải gọi next() để chuyển quả bóng sang  
  middleware tiếp theo  
  next();  
};  
  
module.exports = logRequest;
```

Bước 2: Tạo Validation Middleware (Kiểm tra dữ liệu rác)

Thay vì viết logic kiểm tra if (!name)... trong Controller, ta tách ra middleware riêng.

File: src/middleware/validationMiddleware.js

```
const validateCourseData = (req, res, next) => {  
  const { name, price } = req.body;  
  
  if (!name || name.length < 5) {
```

```

        return res.status(400).json({ message: "Tên khóa học
phải dài hơn 5 ký tự" });
    }

    if (!price || isNaN(price) || Number(price) < 0) {
        return res.status(400).json({ message: "Giá tiền không
hợp lệ" });
    }

    next();
};

module.exports = validateCourseData;

```

2. Cách ghép nối nhiều Middleware (Chaining)

Bây giờ chúng ta sẽ áp dụng cả 3 lớp bảo vệ vào Route **Thêm mới khóa học**.

File: src/routes/courseRoutes.js

```

const express = require('express');
const router = express.Router();

const { createCourse } =
require('../controllers/courseController');

// Import các middleware
const { protect } = require('../middleware/authMiddleware'); //
Cửa kiểm tra Token
const logRequest = require('../middleware/logMiddleware'); //
Cửa ghi Log
const validateCourseData =
require('../middleware/validationMiddleware'); // Cửa kiểm tra
dữ liệu

```

```
// --- CHAINING MIDDLEWARE ---  
  
// Thứ tự chạy: logRequest -> protect -> validateCourseData ->  
createCourse  
  
router.post(  
  '/',  
  logRequest,          // 1. Ghi log trước  
  protect,             // 2. Kiểm tra đăng nhập  
  validateCourseData, // 3. Kiểm tra dữ liệu input (User gửi  
lên)  
  createCourse         // 4. Nếu qua hết 3 của trên mới được  
vào Controller xử lý  
);  
  
module.exports = router;
```

3. Tại sao thứ tự (Order) lại quan trọng?

"Tại sao lại đặt logRequest trước protect?"

- **Nếu logRequest đứng trước:** Dù người dùng đăng nhập sai (bị protect chặn), hệ thống vẫn ghi lại được log là "Có ai đó vừa cố gắng truy cập". Điều này tốt cho việc theo dõi hacker.
- **Nếu protect đứng trước:** Nếu đăng nhập sai, protect sẽ trả lỗi và dừng luôn. logRequest nằm sau sẽ không bao giờ được chạy -> Chúng ta mất dấu vết của request đó.

Nguyên tắc vàng:

1. Các middleware **Global** (Log, CORS, Body Parser) nên đặt trên cùng.
2. Các middleware **Auth** (Xác thực) đặt tiếp theo.
3. Các middleware **Validation** (Kiểm tra dữ liệu) đặt sau Auth.
4. **Controller** đặt cuối cùng.

4. Áp dụng cho toàn bộ ứng dụng (Global Middleware)

Nếu muốn cái logRequest áp dụng cho **tất cả** các API (chứ không chỉ riêng API thêm khóa học), hãy dùng app.use() trong file server.js.

File: server.js

```
const express = require('express');
const logRequest = require('../src/middleware/logMiddleware');
// ... imports khác

const app = express();

app.use(express.json());

// Áp dụng Logger cho TOÀN BỘ hệ thống
// Bất kể gọi API nào cũng sẽ chạy qua hàm này trước tiên
app.use(logRequest);

// Sau đó mới đến các Routes
app.use('/api/courses', courseRoutes);
```

Tổng kết bài học

1. **Luôn nhớ next():** Middleware nào không phải là đích đến cuối cùng (Controller) thì bắt buộc phải gọi next() nếu không muốn Request sẽ không thể tiếp tục.
2. **Sắp xếp thông minh:** Hãy tư duy logic xem cái nào cần chạy trước, cái nào cần chạy sau. (Ví dụ: Phải đăng nhập xong mới kiểm tra quyền Admin -> Middleware Auth chạy trước Middleware CheckAdmin).